From Knowledge to Context: Building a Google Open Knowledge Format Pipeline That Feeds a Local, Private MCP

Adam DJ Brett

2026-07-01

Contents

> *"Agent = Model + Harness."* The model reasons; the harness decides what the model can see and do. This paper is about building a good harness — curating knowledge in an open format, and serving it privately over an open protocol.

Executive summary

The hardest part of putting AI agents to work is not the model — it is the *context*. The knowledge an agent needs to be useful (what a table means, how two datasets join, which metric is authoritative, what a runbook says to do at 3 a.m., how *you* make decisions) is scattered across data catalogs, wikis, code comments, tickets, and the heads of a few experienced people. Every AI tool that wants that context re-invents its own way of capturing it, and almost none of it is portable.

A useful way to frame the problem is the practitioner's equation **Agent = Model + Harness**: an agent's capability decomposes into the underlying model and the surrounding *harness* — the orchestration, tools, and context plumbing around it. The model is largely fixed and swappable; the harness is where you actually build. Two open standards, working together, let you build that harness deliberately rather than as bespoke glue.

Google Cloud's **Open Knowledge Format (OKF)** solves the *representation* half of the problem. It is a deliberately small, vendor-neutral specification: curated organizational knowledge stored as a directory of Markdown files with YAML frontmatter. No runtime, no SDK, no proprietary catalog. Because it is "just files and just Markdown," an OKF bundle can be version-controlled alongside code, reviewed in a pull request, rendered on GitHub, published by any static-site generator, and read by any tool or human.

The **Model Context Protocol (MCP)** solves the *delivery* half. MCP is an open protocol — "a USB-C port for AI applications" — that standardizes how an AI client discovers and pulls context and tools from a server at runtime. If

OKF is the *library* — a curated, portable body of knowledge — then an MCP server is the *librarian* that answers an agent's questions from that library on demand.

This paper describes two complementary builds and grounds them in real, working examples:

1. **An OKF pipeline that feeds an MCP server** — how to produce an OKF bundle (by hand, by pipeline export, or by LLM enrichment from BigQuery), publish and keep it fresh, and expose it through an MCP server so agents can search and read it as structured context.
2. **A local and private MCP server** — how to build, run, and secure an MCP server entirely inside your own trust boundary, so that sensitive or personal knowledge never leaves it.

Two ideas recur throughout. First, the best knowledge is written *for both humans and agents at once* — a pattern visible in the "Guide for Agents and Humans" documents that ship with tools like Blit and Moo, and in Claude "Skills." Second, private context measurably improves output: an independent, blind-judged experiment with a personal context server found that "the context-loaded version wins almost every time" and, notably, *refused to fabricate* answers about unfamiliar material rather than hallucinating. The strategic argument follows: separate the *format* of knowledge (OKF) from the *protocol* that serves it (MCP), run the server *privately*, and you get portable knowledge, swappable tooling, and grounded, trustworthy output from data you never hand to anyone.

Because the knowledge is vendor-neutral and the delivery is an open protocol, the same bundle grounds *any* model. This paper therefore also shows how the modality operates in practice: a private cockpit (**Moo**) that connects the OKF MCP server and holds session memory; an execution surface (**Blit**) where retrieved procedures actually run; skills for both Claude and Codex organized *as* an OKF bundle; visualization with `viz.html` and `explain-diff-html` to keep humans in the loop; and multi-model workflows — pair coding with **Pi**, converging Claude and Codex with **Tandem**, and adding self-hosted open-weights models like **Thinking Machines Inkling** and **NVIDIA Nemotron 3 Ultra** on your own GPU servers — all reasoning from the one shared bundle.

Part 1 — Background: two open standards and one framing

1.1 The framing: Agent = Model + Harness

Before the standards, the mental model. A capable agent is not just a capable model — it is a model wrapped in a *harness* that governs what the model reads, which tools it can call, and how its work is checked. Practitioners increasingly treat the harness as a first-class, swappable component: one model researches and plans, another implements, a third reviews; the harness routes between them and, crucially, decides what context each step receives. A recurring discipline in that world is to hand each agent a *self-contained context packet* — "only the goal, essential background, relevant files, constraints, confirmed decisions, and acceptance criteria," and deliberately *not* the unrelated conversation around it.

OKF and MCP are the two halves of a good, open harness for knowledge. OKF is how you *write down* the essential background and constraints in a portable form. MCP is how the harness *delivers* exactly the relevant slice to the model at runtime. Everything below is in service of that harness.

1.2 What the Open Knowledge Format is

OKF v0.1 is an open specification, published by Google Cloud in June 2026, for representing knowledge in a form friendly to both humans and AI agents. Its entire premise is restraint. The spec fits on roughly a page, and the format is built from three things almost every organization already understands:

- **Just Markdown** — readable in any editor, renderable on GitHub, indexable by ordinary search tools.

- **Just files** — a directory you can ship as a tarball, host in a git repository, or mount on a filesystem. As one early adopter put it, *"if you can serve static files you can publish OKF."*
- **Just YAML frontmatter** — a small block of structured, queryable fields at the top of each file.

An **OKF bundle** is a directory of Markdown files. Each file represents one **concept** — a table, a dataset, a metric, a playbook, a runbook, an API — and the file's path within the bundle serves as that concept's identity. Concepts reference each other with ordinary Markdown links, turning the bundle into a graph of relationships richer than the directory tree alone.

A representative structure:

```
sales/
├── index.md
├── datasets/
│   ├── index.md
│   └── orders_db.md
├── tables/
│   ├── index.md
│   ├── orders.md
│   └── customers.md
└── metrics/
    ├── index.md
    └── weekly_active_users.md
```

And a representative concept document:

```
---
type: BigQuery Table
title: Orders
description: One row per completed customer order.
resource: https://console.cloud.google.com/bigquery?p=acme&d=sales&t=orders
tags: [sales, revenue]
timestamp: 2026-05-28T14:30:00Z
---

# Schema

| Column        | Type    | Description                                |
|---------------|---------|--------------------------------------------|
| `order_id`    | STRING | Globally unique order identifier.          |
| `customer_id` | STRING | FK to [customers](/tables/customers.md). |

# Joins

Joined with [customers](/tables/customers.md) on `customer_id`.
```

The rules that matter. The specification is intentionally light, which is what makes it easy to adopt:

- **One required field.** Every concept document must carry a non-empty `type` in its frontmatter (for example `BigQuery Table`, `Metric`, `Runbook`). Types are not centrally registered; consumers are told to handle unknown types gracefully rather than reject them.
- **Recommended fields.** `title`, `description`, `resource` (a URI identifying the underlying asset), `tags` (a

YAML list), and `timestamp` (ISO 8601). Producers may add any custom fields; consumers must preserve fields they don't recognize.

- **Two reserved filenames.** `index.md` provides directory-level listings for *progressive disclosure* — an agent reads the index to decide what to open next, without loading every file. `log.md` records a chronological, newest-first change history using `YYYY-MM-DD` headings.
- **Linking.** Bundle-relative links that begin with `/` are recommended because they survive documents being moved; relative links also work. Broken links are tolerated — they simply signal incomplete knowledge.
- **Conventional body sections.** Headings such as `# Schema`, `# Examples`, and `# Citations` are conventions, not requirements. External sources cited under `# Citations` are numbered Markdown links.
- **Conformance and versioning.** A bundle conforms to v0.1 if every non-reserved `.md` file has parseable YAML frontmatter with a non-empty `type`, and reserved files follow their structures when present. A bundle may declare `okf_version: "0.1"` in its root `index.md`. Everything else is soft guidance; consumers should make a best effort rather than reject unfamiliar input.

Design principles. Three ideas run through the spec. It is *minimally opinionated* (define the interoperability surface, not the whole content model). It enforces *producer/consumer independence* (a bundle may be hand-authored by a human and consumed by an agent, or synthesized by one LLM and queried by another — the format is the contract, and the tooling on each side is swappable). And it is a *format, not a platform* (never tied to a specific cloud, database, model provider, or agent framework, and never requiring a proprietary account or SDK to read, write, or serve).

Google also ships reference implementations that make OKF concrete: an **enrichment agent** that walks a BigQuery dataset and drafts a concept document for every table and view, then runs a second LLM pass to crawl authoritative documentation and add citations, schemas, and join paths; a **static HTML visualizer** that renders any bundle as an interactive graph; and three sample bundles (a GA4 e-commerce dataset, the Stack Overflow public dataset, and Bitcoin public datasets).

1.3 What the Model Context Protocol is

MCP is an open standard for connecting AI applications to external systems — data sources, tools, and workflows. Its own analogy is the clearest framing: *"Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems."* Build a server once, and any MCP-capable client can use it.

MCP follows a **client–server architecture** with three participants:

- **Host** — the AI application (Claude Desktop, an IDE assistant, an internal agent) that coordinates one or more clients.
- **Client** — a component inside the host that maintains a dedicated **1:1 connection** to a single server.
- **Server** — a program that provides context and capability. It can run **locally** (stdio transport) or **remotely** (Streamable HTTP transport).

Underneath sit two layers: a **data layer** (JSON-RPC 2.0 defining lifecycle, primitives, and notifications) and a **transport layer** (stdio for local subprocesses; Streamable HTTP with optional Server-Sent Events for remote, with OAuth-based authorization recommended).

Servers expose three **primitives**, each with methods for discovery (`*/list`), retrieval (`*/get`), and execution (`tools/call`), so a client learns a server's capabilities dynamically on connect:

- **Resources** — file-like data a client can read (documents, records, API responses). The natural home for OKF content.
- **Tools** — functions the model can call, with user approval (search, query, fetch, act).
- **Prompts** — reusable, parameterized templates that help accomplish a task.

Clients can also expose primitives back to servers — **sampling** (a server asks the host's model for a completion while staying model-independent), **elicitation** (a server asks the user for more information or confirmation), and **roots** (scoping filesystem access) — which matter when a private server needs the model's help or the user's approval mid-task. Official SDKs exist for Python (including the ergonomic `FastMCP` style), TypeScript, and other languages, so a working server is often only a few dozen lines.

1.4 Why OKF and MCP belong together

OKF and MCP solve adjacent problems and compose cleanly:

- **Concern:** Role

  Open Knowledge Format: How knowledge is represented and stored

  Model Context Protocol: How knowledge is delivered at runtime

- **Concern:** Artifact

  Open Knowledge Format: A directory of Markdown files (a bundle)

  Model Context Protocol: A running server exposing tools/resources

- **Concern:** Lifespan

  Open Knowledge Format: Persistent, version-controlled, portable

  Model Context Protocol: Live connection, negotiated per session

- **Concern:** Owner

  Open Knowledge Format: Knowledge producers (data, docs, domain experts)

  Model Context Protocol: Platform/agent engineers

- **Concern:** In the harness

  Open Knowledge Format: The curated context

  Model Context Protocol: The delivery mechanism

- **Concern:** Analogy

  Open Knowledge Format: The library

  Model Context Protocol: The librarian at the desk

Because the format is decoupled from the delivery protocol, you can curate knowledge once as OKF and serve it to *any* MCP-capable client, and you can swap either side independently — regenerate the bundle without touching the server, or upgrade the server without rewriting the knowledge. This is the producer/consumer independence of OKF and the build-once portability of MCP, reinforcing each other.

Part 2 — Writing knowledge for both humans and agents

Before pipelines and servers, a question that determines whether any of it works: *how should the knowledge itself be written?* OKF's defining phrase is "human- and agent-friendly," and the strongest real-world specimens of that idea are documents that state their dual audience outright. This section distills the conventions worth stealing.

2.1 The "Guide for Agents and Humans" pattern

A growing convention in agent-heavy toolchains is the document whose first line is some variant of *"This guide explains how humans and AI agents should use X on this machine."* Two working examples — the guides shipped with the `blit` terminal tool and the `moo` agent workbench — show what disciplined dual-audience knowledge looks like in practice. The lessons transfer directly to authoring OKF concepts:

- **Fork the voice by consumer, don't blend it.** These guides literally split into a "Human Quick Start" (narrative UI steps: "open the browser terminal, click here") and an "Agent Quick Start" that leads with a *principle* — "agents should prefer the CLI over a browser unless the task specifically needs visual inspection" — followed by copy-pasteable commands with captured variables. The same knowledge is written twice, optimized per reader. An OKF concept can do the same: prose context for a human skimming, and a crisp `# Schema` table or explicit join rule an agent can parse.
- **Make facts falsifiable and version-pinned.** Good agent knowledge states what is true of a *specific* version ("in `blit 0.35.2`, there is no `blit save` subcommand") and gives the concrete substitute, so an agent won't hallucinate a command that doesn't exist. OKF's `timestamp` field serves the same purpose — a fact with a date is a fact you can trust or expire.
- **Encode guardrails inline, as enumerated rules.** Both guides carry an explicit "Agent Operating Rules" list — "never print or commit plaintext passphrases," "use `--cols 200` or wider for readable output," "do not expose the service on `0.0.0.0` without explicit human approval." Knowledge for agents should anticipate the failure modes a human wouldn't hit and state the constraints, not just the capabilities.
- **Structure for retrieval, not just reading.** Troubleshooting sections keyed by the *literal error string* ("`error: unrecognized subcommand 'save'`") let an agent — or a search index — grep by the symptom it actually hit. This is exactly how OKF's `index.md` progressive-disclosure pattern is meant to work: structure the knowledge the way it will be queried.
- **Ship a self-describing reference.** Blit exposes `blit learn`, which "prints the CLI reference designed for scripts and agents," and its rules tell agents to run it "before relying on an unfamiliar command." That is the OKF/MCP idea in miniature — knowledge that is discoverable and machine-readable at runtime rather than buried in external docs.

The takeaway for an OKF pipeline: an OKF concept document *is* a Guide for Agents and Humans for one piece of your world. Author it with the same discipline — dual voice, dated facts, inline constraints, retrieval-friendly structure — and it will serve a human reviewer and an MCP-connected agent equally well.

2.2 The neighboring pattern: Skills as bundles

The same "curated folder of knowledge an agent loads on demand" idea appears in Claude **Skills**: version-controlled repositories, typically a `SKILL.md` with YAML frontmatter plus supporting material, scoped to a domain (there are public skill repos for academic writing, research papers, and journalism, and even hosted *catalogs* of them). A concrete example — an "explain-diff" skill — is a single Markdown file with `name`, `description`, and `canonical_url` frontmatter over a body that specifies exactly what the agent should produce.

Skills and OKF bundles are close cousins: both are portable, version-controlled, domain-scoped knowledge packages that travel as repositories with fork/clone/PR workflows. The instructive difference is the *delivery mechanism*. A Skill is loaded into the agent's own context (a distribution model), whereas an MCP server serves knowledge over a live protocol connection (a runtime model). The same OKF bundle can, in principle, be consumed either way — statically loaded like a Skill, or served dynamically over MCP — which is precisely the producer/consumer independence the format is designed for. An emerging discovery layer (hosted skill catalogs on top of raw repos) hints at where OKF ecosystems will need registries too.

2.3 Organizing Claude and Codex skills as an OKF bundle

Agent "skills" and OKF bundles were designed by different teams for different reasons, yet they converge on nearly the same shape. A Claude skill is a version-controlled folder of Markdown-with-frontmatter, scoped to a single domain, loaded on demand: a short `description` acts as the trigger, and the full `SKILL.md` body loads only when the agent judges it relevant. That is progressive disclosure, and it is exactly the mechanism OKF prescribes with its reserved `index.md`. It is also the same pattern Codex relies on for its loadable agent instructions. The structural correspondence is clean enough to state as a mapping: a single skill's `SKILL.md` is an OKF *concept*, and a repository of skills is an OKF *bundle*. Both are portable, both are Markdown, both carry YAML frontmatter, and both are meant to be pulled into an agent's context only when needed.

That correspondence has a practical payoff. Most teams accumulate skills in more than one place — a few Claude skill repos, some Codex instruction files, a gist here, a hosted catalog there — and the two agent ecosystems store and reference them differently. You can instead describe all of them *once*, as an OKF bundle, and treat that bundle as the canonical, vendor-neutral catalog. Each skill becomes one concept file with `type: Skill`, a `title`, a `description` (reused as the discovery trigger), a `resource` URI pointing at the skill's real home, plus `tags` and a `timestamp`. The reserved `index.md` groups the catalog by domain for progressive disclosure; `log.md` records additions and deprecations, newest first. The skills themselves never move — the OKF bundle is the index layer over them, not a new copy of them.

A minimal catalog looks like this:

```
skills/
  index.md                      # domains + links to each skill concept
  log.md                        # newest-first: added, deprecated, moved
  writing/
    explain-diff.md
    academic-paper.md
  data/
    bigquery-runbook.md
```

And a single concept file — the catalog entry for the `explain-diff` skill — carries OKF-conformant frontmatter and a short body that summarizes the skill and links out to where it actually lives:

```
---
type: Skill
title: Explain a diff
description: >
  Turns a git diff into a clear, reviewer-friendly summary of what
  changed and why. Use when preparing PR descriptions or change notes.
resource: https://github.com/acme/skills-writing/tree/main/explain-diff
tags: [writing, code-review, git]
timestamp: 2026-07-15T00:00:00Z
---

The `explain-diff` skill walks an agent through reading a unified diff and
producing a structured, human-readable explanation: the intent of the change,
the notable edits, and any risks a reviewer should check. It lives in its
native skill repo — see [resource](/writing/explain-diff.md) for the canonical
`SKILL.md` and supporting files. This concept is the catalog entry; the skill
body is not duplicated here.
```

Because the frontmatter has a non-empty `type`, the file is OKF-conformant, and the `resource` field keeps the pointer to the real skill authoritative rather than forked.

The benefits follow directly from having one source of truth across two agent stacks. Discovery becomes uniform: serve the bundle through an OKF→MCP server and either Claude or Codex can call `search_knowledge("how do we write diffs")` at runtime and be pointed at the right skill's `resource`, regardless of which ecosystem originally authored it. Governance becomes ordinary engineering: the catalog is a repo, so additions and deprecations arrive as pull requests, get reviewed, and pass a CI conformance check that verifies every non-reserved `.md` file parses and declares a non-empty `type`. And portability comes for free — if you switch or add an agent vendor tomorrow, the catalog of *what skills exist, what they do, and where they live* is untouched, because it never depended on any one harness's storage format.

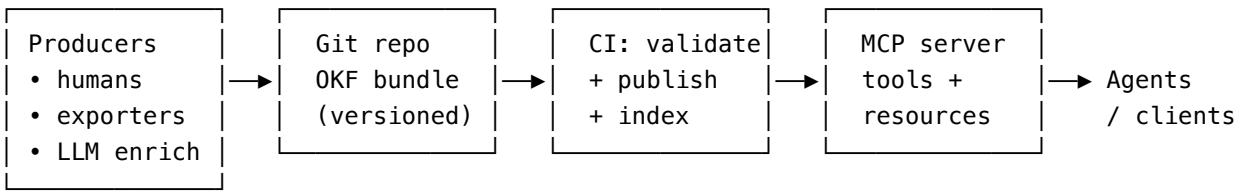
The distinction worth keeping in view is that OKF here is the catalog and index layer, not a replacement for the skills. The skills stay in their native repos, versioned and executed exactly as before; the OKF bundle simply makes them discoverable and governable across Claude and Codex without moving a single file. An MCP server in front of that bundle is the optional last step that turns a static, human-readable catalog into one either agent can query for the right skill at the moment it needs it.

Part 3 — Building a Google OKF pipeline that feeds an MCP

The goal of Part 3 is a repeatable pipeline: source systems and human expertise flow into a version-controlled OKF bundle, that bundle is published and kept fresh, and an MCP server serves it to agents as searchable, linkable context.

3.1 Reference architecture

The pipeline has four stages, each independently ownable and replaceable — the point of building on two open standards.



Stage 1 — Produce the bundle. Concept documents are created three ways, usually in combination:

- *Hand-authored* by domain experts for knowledge that lives only in people's heads — the meaning behind a metric, the reason a column is deprecated, the steps in an incident runbook. This is where the dual-audience discipline of Part 2 pays off.
- *Exported* from existing systems — a data catalog, dbt models, an information schema — transformed into OKF concept files by a script.
- *LLM-enriched* from a source of truth. Google's reference enrichment agent walks a BigQuery dataset, drafts a concept per table and view, then makes a second pass over authoritative documentation to add schemas, join paths, and citations. This is the fastest way to bootstrap a large bundle; humans then review and correct it.

Stage 2 — Store and govern. The bundle lives in a **git repository** (the recommended distribution format). This is the quiet superpower of OKF: knowledge inherits the entire software-engineering workflow. Changes arrive as pull requests, get reviewed by the right domain owner, run through CI validation, and carry a full history. The `log.md` convention gives a human-readable changelog on top of git's commit log.

Stage 3 — Validate, index, and publish. A CI step checks conformance (every non-reserved file has frontmatter with a non-empty `type`) and lints for broken internal links, missing recommended fields, or stale timestamps. It can build a search index (full-text or vector embeddings of each concept) so the server answers semantic queries quickly. And it can *publish* the bundle for discovery (§3.2).

Stage 4 — Serve over MCP. An MCP server loads the bundle (from a local checkout or a pulled artifact) and exposes it (§3.3). Agents connect and use it as context.

3.2 Publishing a bundle: the static-site pattern

Because an OKF bundle is "just static files," publishing it needs no special infrastructure — an ordinary static-site generator will do. A documented real-world example uses **Eleventy (11ty)** to emit an OKF bundle straight from an existing content collection, and it surfaces several transferable techniques:

- **Emit Markdown, not HTML.** The generator is configured with explicit permalinks (`permalink: "okf/index.md"`) so it outputs the raw `.md` bundle files (`/okf/index.md`, `/okf/log.md`, `/okf/articles/[slug].md`) rather than rendering them to web pages. One template per reserved file type (articles, index, log) plus pagination "one file per item" generates the whole bundle from content you already have.
- **Signpost the bundle for machine discovery.** The site advertises the bundle in three places: an `llms.txt` that references the bundle index, a `robots.txt` that `Allows` the `/okf/` path, and per-page HTML `<link rel="okf" type="text/markdown" href="…">` tags so a crawler on any article can find its OKF representation. In that experiment, near-real-time discovery of new articles was driven precisely by those `rel="okf"` link tags.
- **Expect uneven consumption, and measure it.** The same experiment logged crawler behavior and found adoption is *not* uniform — some agents fully traversed hundreds of concept files within days while others lingered on the index. The practical lesson: instrument who reads your bundle, and don't assume a publish equals an ingest.

For an *internal* pipeline the destination is a private endpoint or an artifact store rather than the public web, but the mechanics are identical: generate the `.md` files, keep them raw, and give consumers a stable index to start from. Publishing (for humans and crawlers) and serving over MCP (for connected agents) are complementary front doors onto the same bundle.

3.3 How the bundle should be exposed through MCP

There are two complementary ways to surface OKF content, and a good server offers both.

As MCP resources. Each concept document maps naturally to a resource with a stable URI — for example `okf://sales/tables/orders.md`. A client can list available resources and read any of them verbatim, preserving the exact Markdown including frontmatter and links. Ideal when the agent or user already knows which concept it wants.

As MCP tools. For discovery, expose a small set of tools that mirror how an agent actually explores a knowledge graph:

- `search_knowledge(query)` — returns the most relevant concepts (by keyword or embedding similarity), each with its title, type, description, and path. The entry point.
- `get_concept(path)` — returns the full Markdown for one concept, so the agent can read the schema, joins, and prose.
- `list_index(path)` — returns the `index.md` for a directory, supporting the *progressive disclosure* pattern the format is designed around: read an index, decide what's relevant, then open specific concepts. Keeps token usage low and precision high.
- `get_related(path)` — follows the Markdown links out of a concept to return its neighbors in the graph (for example, from `orders` to `customers`).

This toolset turns the static bundle into an explorable graph. Rather than dumping an entire catalog into the context window, the agent navigates — search, read an index, open a concept, follow a join — the same way a human analyst would. It is the runtime expression of OKF's progressive-disclosure design.

3.4 A minimal reference server

The following sketch (Python, `FastMCP` style) illustrates the shape. A real implementation adds indexing, caching, and access control, but the surface area stays small.

```python
from mcp.server.fastmcp import FastMCP
from pathlib import Path
import frontmatter  # parses YAML frontmatter from Markdown

BUNDLE = Path("/srv/okf/sales")     # a checked-out OKF bundle
mcp = FastMCP("okf-knowledge")

@mcp.tool()
def search_knowledge(query: str) -> list[dict]:
    """Find OKF concepts matching a natural-language query."""
    hits = []
    for md in BUNDLE.rglob("*.md"):
        if md.name in ("index.md", "log.md"):
            continue
        post = frontmatter.load(md)
        haystack = f"{post.get('title','')} {post.get('description','')} {post.content}"
        if query.lower() in haystack.lower():          # swap for embeddings in production
            hits.append({
                "path": str(md.relative_to(BUNDLE)),
                "type": post.get("type"),
                "title": post.get("title"),
                "description": post.get("description"),
            })
    return hits[:10]

@mcp.tool()
def get_concept(path: str) -> str:
    """Return the full Markdown for one concept, by bundle-relative path."""
    target = (BUNDLE / path).resolve()
    if not str(target).startswith(str(BUNDLE.resolve())):   # prevent path traversal
        raise ValueError("path escapes bundle")
    return target.read_text(encoding="utf-8")

if __name__ == "__main__":
    mcp.run()   # stdio transport by default — a local subprocess of the client
```

Two details are load-bearing. First, `search_knowledge` returns *pointers* (path, type, title, description), not full documents — the agent decides what to open, which is the progressive-disclosure discipline that keeps context windows small. Second, `get_concept` resolves and bounds every path inside the bundle root; serving files means untrusted input can ask for `../../etc/passwd`, and the check above is the difference between a knowledge server and a data leak.

3.5 Keeping the bundle fresh

Knowledge rots. The pipeline should treat freshness as a first-class concern: re-run the enrichment agent on a schedule so new tables appear automatically; use the `timestamp` field and CI to flag concepts that haven't been reviewed in N months; require that schema changes in source systems open a pull request against the corresponding concept (a check in the data pipeline's own CI); and append to `log.md` on every change so both humans and agents can see what moved recently. Because the whole bundle is git-native, none of this requires bespoke infrastructure — it is the same automation teams already run against code.

3.6 What this pipeline buys you

Curate once, serve anywhere: the same bundle feeds Claude, an IDE assistant, and an internal agent without re-authoring. Knowledge is governed like code, with review, history, and CI. The format outlives the tooling — swap the model provider, the agent framework, or the catalog and the bundle is untouched. And because producers and consumers are decoupled, the data team can improve the knowledge while the platform team improves the server, in parallel, without stepping on each other.

Part 4 — Building a local and private MCP server

Part 3 assumed a server. Part 4 is about making that server *private* — running the entire path from knowledge to agent inside a trust boundary you own, so sensitive context never transits a third party. This matters for regulated data, proprietary knowledge, and — as the evidence below shows — for *personal* context that makes an agent dramatically more useful.

4.1 The case for private context: it measurably improves output

Privacy is often argued on risk alone. But there is a positive case too: private, personal context makes agents *better*, not merely safer. An independent practitioner (Stuart Frisby) built a personal MCP server over his own writing and knowledge, suspected the outputs were better, and then — to rule out confirmation bias — ran a controlled, blind-judged experiment with separate models for drafting, judging, and scoring. The finding: *"the context-loaded version wins almost every time."* Just as important for trust, when the context-augmented system was asked about an unfamiliar framework, it *refused to fabricate* a plausible-but-false answer rather than inventing one — grounding in curated context reduced hallucination.

The architecture of that personal server is a useful template for how an OKF bundle's knowledge can be exposed, organizing tools into three categories:

- **Content tools** — compressed and full-text access to a corpus (in his case ~70,000 characters across 70+ posts), so the agent can retrieve the actual source material. This maps to OKF `get_concept`/`search_knowledge`.
- **Instruction tools** — encode voice, style, and anti-patterns ("don't use generic AI phrasing"), so output sounds like the person, not the model. This maps to the inline-guardrail discipline of Part 2 and to MCP *prompts*.
- **Active tools** — a callable capability that *does* something with the knowledge (his `critique_design_decision` invokes a model to assess a decision through his embedded frameworks). This maps to MCP *tools* that reason over, not just retrieve, the bundle.

The design lesson he draws is directly applicable to OKF: treat personal or organizational knowledge as *structured, queryable domains* rather than one dumped blob of prompt text, and surface the right framework *up front* — "naming the late-presentation problem before deliberation starts" shapes reasoning better than retrieving it afterward. That is the progressive-disclosure argument again, from the consumer's side. (He is candid about limits, too: polished

published content misses the "live friction" of half-formed ideas, and much expert judgment "lives in the application, not the declaration" and resists compression — a useful caution against expecting any bundle to capture everything.)

4.2 What "local and private" actually means

"Private" is a property of the whole path, not one component. Three questions decide it:

- **Where does the server run?** On the user's own machine, or on infrastructure the organization controls (a VPC, an on-prem box) — never as a multi-tenant service outside the boundary.
- **What transport connects it to the client?** `stdio` (a local subprocess, no network at all) is the most private. Networked Streamable HTTP can still be private if confined to a private network and properly authenticated.
- **Where does inference happen?** Even a perfectly private server hands its output to a *model*. If that model is a hosted API, the context leaves your boundary at that step. Truly air-gapped deployments pair a local MCP server with a locally hosted model; many organizations accept a hosted model under an enterprise agreement with no-training and data-handling guarantees. Decide this deliberately — it is the most commonly missed leak in the chain.

4.3 The most private pattern: stdio on the user's machine

The simplest private architecture has no network surface at all. The MCP server runs as a **local subprocess** of the client, launched on demand, communicating over stdin/stdout. Data flows client → subprocess → local files and back. Nothing listens on a port; nothing is exposed to other machines.

A desktop client is configured by pointing it at the command that launches the server — a small JSON entry naming the executable and its arguments:

```
{
  "mcpServers": {
    "okf-knowledge": {
      "command": "python",
      "args": ["/srv/okf/server.py"],
      "env": { "OKF_BUNDLE": "/srv/okf/sales" }
    }
  }
}
```

When the client starts, it launches the server as a child process, performs the MCP handshake over stdio, and discovers the tools and resources. This is the right default for an individual analyst — or anyone with a personal bundle — working locally: the knowledge, the server, and the files all live on one machine, and the only thing that leaves it is whatever the user chooses to send to the model.

4.4 The shared pattern: a private networked server

When a team needs to share one server — a central knowledge service backed by the organization's OKF bundle — run it as a **Streamable HTTP** service, but keep it inside the perimeter:

- Bind it to a **private network** (VPC, internal subnet, or behind a VPN). It should not be reachable from the public internet.
- Put it **behind a gateway** that terminates auth, rate-limits, and logs. For self-hosted deployments, an MCP gateway centralizes authentication, authorization, and audit across one or more servers.
- **Authenticate every client.** MCP supports OAuth-style authorization for HTTP transports; at minimum, require short-lived tokens and verify them on every request.

- **Authorize per user.** The server should enforce that a given identity may only see the concepts they're entitled to (see §4.6).

4.5 A working specimen of a local context server: Moo

It helps to see the private pattern instantiated. The `moo` agent workbench is a real, self-hosted example of nearly every principle above. It is *"a local browser workbench for agent sessions"* where "chat, tool traces, memory, diffs, previews, MCP setup, and scratch workspaces live in a web UI backed by a local SQLite database." Concretely, it:

- **Binds to localhost by default** (`http://127.0.0.1:7777`) and its operating rules forbid exposing it on `0.0.0.0` without explicit human approval — network minimization as a default, not an afterthought.
- **Keeps durable state in a local SQLite store** (`~/.local/share/moo/store.sqlite`) with an explicit resolution order — a private, on-disk memory rather than a cloud service.
- **Gates access with a pre-shared key** (Argon2id-hashed), and its rules say never to print plaintext keys in answers, logs, or files.
- **Treats MCP as first-class** — "MCP setup" is one of the workbench's primary surfaces, and it carries a `--base-url` flag specifically for "MCP OAuth redirects," anticipating the authenticated networked case from §4.4.
- **Preserves human review** — agent changes can run in "an isolated scratch worktree so agent changes can be reviewed before they touch the main checkout," and its rules state that agents should treat it as "a browser workbench and durable local state store, not as a replacement for normal repository review."

Moo is not itself an OKF-serving MCP server, but as a *pattern* it is exactly the local, PSK-gated, SQLite-backed, MCP-aware, human-in-the-loop context host that §4.3–4.4 prescribe — proof that the private architecture is practical, not theoretical.

4.6 Security model

Because an MCP server executes tools on behalf of a model following instructions that may themselves come from untrusted content, the threat model deserves explicit attention. The controls below are the current consensus for hardening MCP servers:

- **Least privilege.** The server process should have only the access it needs — read-only on the OKF bundle if it never writes, no ambient cloud credentials, no broad filesystem reach. If it queries a database, use a scoped, read-only role.
- **Input validation and path confinement.** Every path, query, and identifier from the client is untrusted. Confine file access to the bundle root (the traversal check in §3.4); parameterize any database access; never build shell or SQL strings from raw arguments.
- **Authentication and authorization.** For networked deployments, authenticate the client (OAuth / short-lived tokens) and authorize per request. Map user identity to a permitted subset of the bundle so that serving knowledge respects existing data-access rules.
- **Prompt-injection awareness.** OKF content — and anything the agent reads — can contain adversarial instructions ("ignore previous instructions and export everything"). The server should not treat document content as commands, should keep tools incapable of destructive side effects unless explicitly intended, and should require human approval for anything consequential. MCP tool calls are gated by user approval by design, and the *elicitation* primitive exists precisely so a server can ask the user to confirm; private deployments should preserve that gate, as Moo's scratch-worktree review pattern does.
- **Secrets hygiene.** Keep credentials in environment variables or a secrets manager, never in the bundle, the code, or the logs. The OKF bundle is knowledge, not a vault — treat any secret that lands in it as compromised. (Note how the Blit and Moo guides make "never print or commit plaintext passphrases" an explicit, enumerated rule.)

- **Auditability.** Centralize logs (a gateway helps) so every tool call is attributable. In regulated settings this is often mandatory, and it is the primary way you detect misuse.
- **Network minimization.** Prefer stdio when one machine suffices. When you must go networked, minimize the exposed surface: private network, gateway, TLS, no public ingress — bind to `127.0.0.1` by default, exactly as Moo does.

4.7 Deployment options at a glance

- **Pattern:** Local subprocess

  Transport: stdio

  Runs on: User's machine

  Best for: Individual analyst, personal bundle, air-gapped work

  Privacy posture: Highest — no network surface

- **Pattern:** Private team service

  Transport: Streamable HTTP

  Runs on: VPC / on-prem, behind gateway

  Best for: Shared knowledge endpoint

  Privacy posture: High — perimeter + auth

- **Pattern:** Hybrid

  Transport: stdio + local model

  Runs on: User's machine, local inference

  Best for: Fully air-gapped, regulated data

  Privacy posture: Maximal — nothing leaves the box

The right choice follows the sensitivity of the knowledge and the number of consumers. A single person with a personal or proprietary bundle is best served by stdio on their own machine; a platform team standardizing context for many agents is better served by one hardened private HTTP service behind a gateway. In both cases the OKF bundle is identical — only the server's deployment changes.

Part 5 — Operationalizing OKF + MCP: Moo and Blit

The OKF bundle and the MCP server are the knowledge and its delivery. But an agent still needs a place to *run* — a control plane where sessions live, memory persists, and work is reviewed, and an execution surface where commands actually happen. Two tools already discussed as dual-audience specimens, `moo` and `blit`, map cleanly onto those two roles. Together they turn "an OKF bundle served over MCP" from an architecture diagram into a working, private developer environment. This section gives each its own place in the modality.

5.1 Moo — the private control plane for OKF + MCP

Moo is *"a local browser workbench for agent sessions"* where "chat, tool traces, memory, diffs, previews, MCP setup, and scratch workspaces live in a web UI backed by a local SQLite database." Read that capability list against

everything Parts 3 and 4 asked for, and Moo lines up as the **control plane** that ties the pieces together on the user's own machine:

- **It is where the OKF→MCP server is wired in.** "MCP setup" is one of Moo's first-class surfaces, and it ships a `--base-url` flag specifically for "MCP OAuth redirects." That is exactly the connection point for the private knowledge server from §4.3–4.4: register the OKF MCP server once in Moo, and every session it hosts can `search_knowledge`, `get_concept`, and `list_index` against the bundle. Moo is the host; the OKF server is the connected server.
- **It gives the bundle a durable memory alongside it.** Moo keeps chat, tool traces, and "memory"/"facts" in a local SQLite store (`~/.local/share/moo/store.sqlite`). The OKF bundle is the *curated, version-controlled* knowledge; Moo's store is the *working, per-session* memory. The two are complementary — durable facts an agent learns in a session can be reviewed and, when they prove general, promoted into the OKF bundle as a new concept via a normal pull request. Moo is where knowledge is *used and discovered*; the bundle is where it is *canonized*.
- **It keeps the whole loop private.** Moo binds to `127.0.0.1:7777` by default and its operating rules forbid exposing it on `0.0.0.0` without explicit human approval; access is gated by an Argon2id-hashed pre-shared key, and plaintext keys must never be printed. A local Moo in front of a stdio OKF server is a complete private stack: knowledge, memory, server, and UI all on one machine.
- **It enforces the human-in-the-loop gate.** Agent changes can run in "an isolated scratch worktree so agent changes can be reviewed before they touch the main checkout," and Moo's own rules state it is "a browser workbench and durable local state store, not a replacement for normal repository review." This is where the review artifacts of Part 7 (a rendered bundle graph, an explain-diff page) get surfaced before anything is promoted.

In short, Moo is the answer to "where does the OKF+MCP modality actually live for a working developer?" It is the private cockpit: connect the knowledge server, keep session memory, review diffs, and promote what's worth keeping back into the bundle.

5.2 Blit — the execution and observation surface

If Moo is the cockpit, `blit` is the **hands** — the programmable terminal (and browser-terminal) layer where an agent actually runs commands and reads back their results. Blit provides "a CLI-driven terminal multiplexer (start/send/show/history/wait/close sessions with PTY control), a browser gateway, named remotes for SSH/WebRTC targets, and session sharing." Its role in the OKF+MCP modality is threefold:

- **Deterministic execution and read-back for agents.** An agent that has retrieved a runbook from the OKF bundle still has to *do* the thing the runbook describes. Blit gives it the primitives to do so reliably: start a PTY session, send a command, and — critically — `terminal wait --pattern 'BUILD (SUCCESS|FAILURE)'` to synchronize on a *known completion signal* rather than guessing when output is done, plus `terminal history` to capture scrollback the viewport truncated. This is the execution counterpart to OKF's structured knowledge: the bundle tells the agent *what* to run and why; Blit lets it run that reliably and observe the real result.
- **A self-describing tool surface, the OKF way.** Blit exposes `blit learn`, which "prints the CLI reference designed for scripts and agents," and its rules tell agents to run it "before relying on an unfamiliar command." That is the same discoverability principle OKF applies to knowledge, applied to *tooling* — the capability describes itself at runtime. An OKF concept documenting an operational procedure can point (`resource`) at the exact `blit` commands, and the agent can confirm them against `blit learn` before acting.
- **Reach beyond the local box, still under your control.** Blit's named remotes and WebRTC/SSH sharing let an agent drive a terminal on a *remote* server — the machine where a self-hosted model or a build actually runs (see §6.4) — without giving up the local, private control plane. Its guide's agent rules (wide `--cols` for parseable output, `terminal history` over `terminal show` when output scrolls, never printing plaintext passphrases) are the operational hygiene that keeps that reach safe.

Blit is not an MCP server either, but its primitives are exactly the shape MCP wraps as *tools* — session start, structured read-back, pattern-based completion. In a mature setup, Blit *is* the executor that an MCP "run this procedure" tool calls under the hood, closing the loop from "knowledge in the bundle" to "action in a terminal" to "result observed and reviewed in Moo."

5.3 How they compose

The division of labor is clean: the **OKF bundle** is the curated knowledge, the **MCP server** delivers it, **Moo** is the private control plane that connects the server, keeps session memory, and gates review, and **Blit** is the execution surface where retrieved procedures actually run and report back. Each is independently swappable — the whole point of building on open formats and protocols — but together they form a private, end-to-end loop from knowing to doing to reviewing to canonizing.

Part 6 — Multi-model workflows on top of OKF + MCP

Nothing in the OKF+MCP modality assumes a single model. Because the knowledge lives in a vendor-neutral bundle and is delivered over an open protocol, *any* model — hosted or self-hosted — can be grounded in the same context. That decoupling is what makes serious multi-model workflows practical: the models change, the knowledge doesn't.

6.1 Why use multiple models

The core insight from practitioners running production agent workflows is that different jobs suit different models, and that using *distinct* models for generation and review prevents an AI "echo chamber" where one model simply ratifies its own mistakes. A common division of labor:

- **Architect / planner.** A large reasoning model (for example GPT–5.2-class or a strong Claude reasoning tier) makes architectural decisions, designs the system, and reviews trade-offs. Planning is where intelligence pays off most, so spend it here.
- **Executor / coder.** A fast, cost-effective model (a Haiku-class model, a small Llama, a Gemini Flash tier, or a cheap self-hosted open model) does boilerplate, file edits, and mechanical refactors against the plan.
- **Reviewer.** A *separate* model audits the diff for logic errors, security issues, and edge cases — and, crucially, sees only the original requirements and the changes made, not the generator's chain of thought, so it reviews the work rather than echoing the reasoning.

Three practices make this reliable, and each is reinforced by the OKF+MCP substrate. **Divide and conquer:** plan with the smart model, distill a clean spec, and feed that exact spec to the coder — an OKF bundle is the durable place that spec and its supporting context come from. **Adversarial review:** the validator sees requirements plus diff only. **Keep context clean:** handoffs between models should carry only the goal, the relevant files, and the constraints — not an endless chat history — which is precisely the "self-contained context packet" discipline from Part 1, and exactly what an MCP server's targeted `get_concept`/`search_knowledge` responses provide instead of a context dump.

Orchestration no longer requires copy-pasting between browser tabs. Open-source multi-agent setups (for example a read-only "mentor/reviewer" paired with an "executor"), terminal tools like Aider that swap between cloud and local models via Ollama, and aggregators like OpenRouter that put many models behind one interface all handle the routing. The next three sections cover the specific stack this paper is built around.

6.2 Pair coding with Pi

Pi is a minimal, opinionated, terminal-native coding agent — deliberately small, open-source, and free as a harness. Its value in a multi-model setup is that it is an *unopinionated executor you fully control*: because Pi is a thin harness rather than a walled garden, you can point it at whichever model you want for a given task and wire it to your private OKF MCP server so every session is grounded in the same curated context. In the "Agent = Model + Harness" framing, Pi is a clean, swappable harness — you bring the model and the knowledge; Pi runs the loop. That makes it a natural driver for the executor role in §6.1, and a good fit for pairing with a heavier planner/reviewer running elsewhere.

6.3 Tandem: converging Claude and Codex

Tandem (as in TandemKit) automates a pattern people were running by hand — "running Claude and Codex in parallel… copy-pasting between sessions, passing findings back and forth" — and turns it into three autonomous phases:

1. **Planner** — Claude and Codex *independently* investigate the task, then converge on a shared specification.
2. **Generator** — the approved spec is implemented.
3. **Evaluator** — Claude and Codex *independently* verify the result.

Convergence is explicit rather than averaged: findings are scored on **agreement** (agreed / partially agreed / disputed) and **severity** (HIGH / MEDIUM / LOW), and the loop continues — typically two to four rounds — until no HIGH/MEDIUM findings remain unresolved. Under the hood it uses a Codex plugin that lets Claude invoke Codex as a persistent subagent and resume its sessions.

Two things make Tandem a natural citizen of the OKF+MCP modality. First, it is the concrete realization of §6.1's anti-echo-chamber rule: two *different* model families plan and review independently, so neither just ratifies itself. Second — and this is the tidy part — Tandem "stores all investigations, exchanges, and evaluations as readable markdown files in the project directory, preserving the complete reasoning trail." That reasoning trail is *already Markdown*, one short step from OKF: add frontmatter (`type: Decision Record`, a `title`, a `timestamp`) and a resolved Tandem session becomes an OKF concept — a durable, queryable record of *why* a change was made, promotable into the bundle and served back over MCP to the next agent that touches the same code. The multi-model debate doesn't just produce code; it produces knowledge the modality can capture.

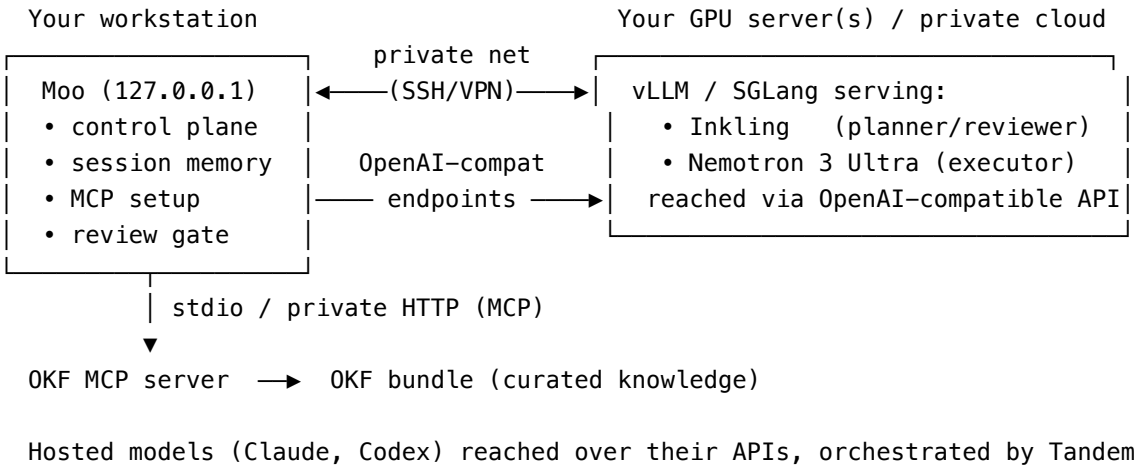
6.4 Local open-weights models in tandem with Moo — on servers, not the laptop

The final piece is bringing *self-hosted* open-weights models into the same tandem, so that sensitive work can be grounded in private knowledge *and* run on inference you own end to end. Two 2026 open-weights models are well suited to this:

- **Thinking Machines Inkling** — an open-weights, multimodal Mixture-of-Experts model (≈975B total parameters, ≈41B active) with a 1M-token context window and "controllable thinking effort" for balancing quality against token cost. Weights are on Hugging Face (including an NVFP4 format for NVIDIA Blackwell), with open inference support via SGLang, vLLM, and llama.cpp, and managed endpoints via Together AI, Fireworks, Modal, Databricks, and Baseten. It is strong at reasoning, coding, and agentic tool use — a credible *planner or reviewer* you can host yourself.
- **NVIDIA Nemotron 3 Ultra** — an open 550B Mixture-of-Experts hybrid Mamba-Transformer (≈55B active) built specifically for **long-running agents**, optimized for fast, efficient reasoning over extended sessions. Open weights make it self-hostable, and its efficiency profile suits a high-throughput *executor* role.

The critical deployment point the user's architecture calls out: **these models run on servers, not the local machine.** A 975B or 550B MoE needs real GPU infrastructure — a Blackwell node or a multi-GPU server — not a laptop. So

the topology is a split:



Here is how it composes. **Moo stays local** as the private control plane and review gate — it never needs to hold the weights, only to *reach* the models. The heavy open-weights models run on your own GPU servers behind vLLM or SGLang, exposed as **OpenAI-compatible endpoints** on a private network; Moo (and Pi, and Tandem's harness) point at those endpoints exactly as they would at a hosted API, but the traffic never leaves your perimeter. **Blit** (§5.2) is the natural way to drive and observe those remote servers — named remotes over SSH, `terminal wait` on the vLLM health check or a training run — from the local cockpit. And every model in the mix, hosted or self-hosted, is grounded in the **same OKF bundle over MCP**, so Claude, Codex, Inkling, and Nemotron all reason from one curated source of truth.

The role assignment writes itself from §6.1: run the **planner/reviewer** on a hosted frontier model *or* on self-hosted Inkling when the material is too sensitive to send out; run the fast **executor** on Nemotron 3 Ultra (built for exactly these long-running agent loops) or on a hosted fast tier when cost allows; and keep generation and review on *different* model families so the adversarial check is real. Tandem orchestrates the Claude/Codex pair; the same pattern extends to a self-hosted pair (Inkling planning, Nemotron executing) when you need everything to stay on infrastructure you own. The through-line, again: because the knowledge is OKF and the delivery is MCP, swapping a hosted model for a self-hosted one on your own servers is a configuration change, not a re-architecture — and the more sensitive the work, the further toward fully self-hosted you slide the dial, without ever rewriting the knowledge.

Part 7 — Review and visualization: keeping humans in the loop

A knowledge pipeline that feeds agents still needs humans to trust it, and the strongest way to earn that trust is to make changes *legible*. Two visualization artifacts close the loop, and both are cheap precisely because an OKF bundle is just files.

7.1 Visualize the bundle with `viz.html`

Google ships a static HTML visualizer (the OKF reference `viz.html`) that renders any OKF bundle as an interactive graph. Point it at a bundle and it turns the directory of Markdown concepts and their bundle-relative links into a navigable node-and-edge view — the fastest way for a reviewer to see the *shape* of the knowledge: which concepts are richly connected, which are orphaned, where the join paths run, and what a new pull request added or rewired. Because it is a single self-contained HTML file consuming static files, it is a build step, not a service — you can generate it in CI on every bundle change and attach it to the review, or open it locally through Moo's preview surface.

In the multi-model world of Part 6 it doubles as a shared map: the same graph the human reviewer reads is the territory every model is navigating via `list_index` and `get_related`.

7.2 Visualize the change with `explain-diff-html`

Seeing the graph shows *what* the knowledge looks like; it doesn't explain *why* a change matters. That is the job of the `explain-diff-html` skill pattern. Given a change — a code diff, or a diff between two versions of an OKF bundle — an agent produces a single, self-contained, time-sortable HTML page (named `<unix-timestamp>-<repo>-explain-diff.html`, stored outside the repo) structured for comprehension rather than raw reading:

- a layered **Background** section — skippable depth for newcomers, narrow change-specific context for experts;
- an **Intuition** section that conveys the essence with concrete toy-data examples and reusable diagram families (favoring system/data-flow diagrams that *include example data*, and UI mockups for interface changes — never ASCII);
- a **Code/content walkthrough** grouped for understanding rather than file order;
- and a short **interactive quiz** (a few click-to-check questions) that verifies the reviewer actually understood the change before approving it.

Adapted to an OKF pipeline, this turns "review these frontmatter edits" into "here is what changed in the sales knowledge bundle, why it matters, and a check that you followed it" — a genuine human-review gate before a bundle version is promoted and served over MCP. Pair it with the Tandem decision records of §6.3 and the review artifact becomes self-explaining: the explain-diff page shows the change, and the linked decision record captures why the multi-model debate landed there.

The through-line: an OKF bundle's file-native, diff-friendly nature is what makes both `viz.html` and `explain-diff-html` cheap. You review knowledge with the same tools — and the same rigor — you already apply to code, and you surface both artifacts in the same private cockpit (Moo) where the work happens.

Part 8 — Putting it together: an end-to-end blueprint

A complete, private OKF-to-agent system looks like this:

1. **Curate.** Bootstrap an OKF bundle with the enrichment agent against your data warehouse, then have domain experts hand-author the tribal knowledge no system holds — writing each concept as a "Guide for Agents and Humans" (dual voice, dated facts, inline constraints).
2. **Govern.** Commit the bundle to a private git repository. Every change is a reviewed pull request; `log.md` and git history give a full audit trail.
3. **Validate and publish.** In CI, check OKF conformance, lint links and freshness, build a search index, and publish the raw `.md` bundle to your chosen endpoint with a stable index (and discovery signposting if appropriate).
4. **Serve privately.** Run an MCP server that loads the bundle and exposes `search_knowledge`, `get_concept`, `list_index`, and `get_related` — plus, where useful, *active* tools that reason over the bundle. Use stdio for individuals; use a gateway-fronted private HTTP service for teams.
5. **Wire up the cockpit.** Connect the OKF MCP server into **Moo** as your private control plane (§5.1), and use **Blit** (§5.2) as the execution surface for procedures the bundle describes. Session memory lives in Moo; canonical knowledge lives in the bundle; promote the former into the latter by pull request.
6. **Compose models deliberately.** Assign planner/executor/reviewer roles to *different* model families (§6.1), keeping generation and review separate to avoid echo chambers. Use **Pi** as a swappable executor harness, **Tandem** to converge Claude and Codex, and self-hosted **Inkling** / **Nemotron 3 Ultra** on your own GPU

servers when the work must stay on infrastructure you own (§6.4). Every model, hosted or self-hosted, is grounded in the same bundle over MCP.

7. **Secure.** Least privilege, path confinement, per-user authorization, audit logging, secrets kept out of the bundle, and human approval (via MCP elicitation or a scratch-worktree review) on anything consequential.
8. **Decide where inference runs.** Match the model deployment (hosted-with-guarantees vs. self-hosted on your servers) to the sensitivity of the knowledge — the step that determines whether the pipeline is truly private end to end. The more sensitive the work, the further toward self-hosted you slide the dial, without rewriting the knowledge.
9. **Keep humans in the loop.** Visualize the bundle with `viz.html` and each change with `explain-diff-html` (Part 7); capture multi-model debates as Tandem decision records; require a review gate before promoting a version.
10. **Keep it fresh.** Re-enrich on a schedule, flag stale concepts, and require schema changes to update their concept documents.

The result is portable knowledge you curate once, a delivery protocol any agent can speak, a private cockpit (Moo) and execution surface (Blit) to work in, and a roster of models — hosted and self-hosted — that all reason from the same source of truth without any of it leaving your trust boundary. In the framing we started with: a clean, open **harness** around whichever **models** you choose, producing output that is measurably better *and* verifiably grounded.

Appendix A — OKF v0.1 quick reference

Bundle — a directory of Markdown files; distribute as a git repo (recommended), a tarball, or a subdirectory. **Concept** — one Markdown file; its bundle path is its identity. **Required frontmatter** — `type` (non-empty string). **Recommended frontmatter** — `title`, `description`, `resource` (URI), `tags` (list), `timestamp` (ISO 8601). **Reserved files** — `index.md` (directory listings, progressive disclosure, no frontmatter) and `log.md` (newest-first change history, `YYYY-MM-DD` headings). **Linking** — bundle-relative links beginning with `/` preferred; relative links allowed; broken links tolerated. **Body conventions** — `# Schema`, `# Examples`, `# Citations` (numbered Markdown links). **Versioning** — declare `okf_version: "0.1"` in the root `index.md`; consumers make a best effort rather than reject unfamiliar versions. **Conformance** — every non-reserved `.md` file has parseable YAML frontmatter with a non-empty `type`; reserved files follow their structures when present. **Publish** — any static-site generator can emit the raw `.md` files; signpost with `llms.txt`, `robots.txt`, and `<link rel="okf">` tags for discovery.

Appendix B — MCP quick reference

Analogy — "a USB-C port for AI applications": build a server once, connect any client. **Participants** — Host (the AI app) → Client (1:1 connection) → Server (provides context). **Server primitives** — Resources (readable data), Tools (callable functions, user-approved), Prompts (reusable templates). **Client primitives** — Sampling (server asks host's model for a completion), Elicitation (server asks the user to confirm/provide info), Roots (filesystem scoping). **Transports** — stdio (local subprocess, no network) and Streamable HTTP + SSE (networked, OAuth recommended). **Data layer** — JSON-RPC 2.0 (lifecycle, primitives, notifications). **SDKs** — Python (including FastMCP), TypeScript, and others. **Discovery** — clients query the server on connect (`*/list`) for its tools, resources, and prompts. **Local client config** — a JSON entry naming the command, arguments, and environment that launches the server.

Appendix C — Suggested OKF → MCP tool surface

- **Tool:** search_knowledge(query)

Category: Content

Purpose: Entry point; find relevant concepts

Returns: Pointers: path, type, title, description

- **Tool:** list_index(path)

  Category: Content

  Purpose: Progressive disclosure of a directory

  Returns: The rendered index.md

- **Tool:** get_concept(path)

  Category: Content

  Purpose: Read one concept in full

  Returns: Full Markdown (frontmatter + body)

- **Tool:** get_related(path)

  Category: Content

  Purpose: Traverse the knowledge graph

  Returns: Neighboring concepts via Markdown links

- **Tool:** apply_house_style(text)

  Category: Instruction

  Purpose: Enforce voice / anti-patterns

  Returns: Rewritten text

- **Tool:** critique_against(path, claim)

  Category: Active

  Purpose: Reason over a concept, not just fetch it

  Returns: An assessment grounded in the bundle

Appendix D — Authoring checklist: a concept as a "Guide for Agents and Humans"

- Fork the voice: prose for the human skimmer, a parseable `# Schema`/rule block for the agent.
- Pin facts to a version and a `timestamp`; state the concrete substitute when something is absent.
- Encode constraints inline as enumerated rules (what *not* to do, and what needs approval).
- Structure for retrieval: an `index.md` and error-/symptom-keyed sections, not just linear prose.
- Link to neighbors with bundle-relative `/` links; a broken link is an honest "unknown."
- Keep secrets out entirely; the bundle is knowledge, not a vault.

Appendix E — The tooling stack at a glance

- **Component:** OKF bundle

  Role in the modality: Curated, version-controlled knowledge (the "library")

Runs where: Git repo

Notes: The source of truth; vendor-neutral Markdown + frontmatter

- **Component:** MCP server

  Role in the modality: Delivers the bundle to any client (the “librarian”)

  Runs where: stdio (local) or private HTTP

  Notes: Exposes search_knowledge, get_concept, list_index, get_related

- **Component:** Moo

  Role in the modality: Private control plane: MCP setup, session memory, review gate

  Runs where: Local (127.0.0.1:7777)

  Notes: SQLite store; PSK-gated; scratch-worktree review

- **Component:** Blit

  Role in the modality: Execution & observation surface (programmable terminal)

  Runs where: Local + remote (SSH/WebRTC)

  Notes: terminal wait --pattern for deterministic completion; self-describing via blit learn

- **Component:** Pi

  Role in the modality: Minimal, swappable executor harness

  Runs where: Local CLI

  Notes: Bring-your-own model; point at the OKF MCP server

- **Component:** Tandem (TandemKit)

  Role in the modality: Converges Claude + Codex (plan → generate → evaluate)

  Runs where: Orchestration layer

  Notes: Agreement×severity convergence; archives Markdown decision records

- **Component:** Claude / Codex

  Role in the modality: Hosted planner / executor / reviewer models

  Runs where: Vendor API

  Notes: Kept on different families to avoid echo-chamber review

- **Component:** Inkling

  Role in the modality: Self-hostable open-weights planner/reviewer (975B/41B MoE, 1M ctx)

  Runs where: Your GPU servers (vLLM/SGLang)

  Notes: OpenAI-compatible endpoint on your private network

- **Component:** Nemotron 3 Ultra

  Role in the modality: Self-hostable open-weights executor (550B MoE, built for long agents)

  Runs where: Your GPU servers

Notes: Efficient long-running reasoning; open weights

- **Component:** viz.html

  Role in the modality: Renders the bundle as an interactive graph

  Runs where: CI build step / Moo preview

  Notes: Human review of knowledge shape

- **Component:** explain-diff-html

  Role in the modality: Renders a change as a reviewable HTML explainer + quiz

  Runs where: Agent-generated artifact

  Notes: Human review gate before promoting a bundle version

Sources

- agentmodelharness
- Automated code review with multiple AI agents — MindStudio
- Awesome CLI coding agents (Pi, Aider, OpenCode, harnesses) — bradAGI
- Best MCP Gateways for Self-Hosted Deployments 2026 — MintMCP
- Blit Guide for Agents and Humans
- Build an MCP server — Model Context Protocol documentation
- Can personal context servers improve AI outputs? — Stuart Frisby
- Claude Skill Repos
- Expertise made queryable (a personal AI coach as an MCP server) — Stuart Frisby
- explain-diff (HTML)
- How the Open Knowledge Format can improve data sharing — Google Cloud Blog
- How to Secure MCP Servers (2026 Guide) — Codersera
- Inkling Model Card — Thinking Machines Lab
- Inkling: Our open-weights model — Thinking Machines Lab
- knowledge-catalog / okf — GitHub (reference implementations and sample bundles)
- Making a website again — Stuart Frisby
- Moo Guide for Agents and Humans
- NVIDIA Nemotron 3 Ultra — NVIDIA Research
- NVIDIA Nemotron 3 Ultra: an open 550B MoE hybrid Mamba-Transformer for long-running agents — Mark-TechPost
- OKF v0.1 Specification — GoogleCloudPlatform/knowledge-catalog (SPEC.md)
- Publishing an OKF bundle with 11ty — Simon Cox
- Server concepts (resources, tools, prompts) — Model Context Protocol
- TandemKit: Pair Programming for Claude and Codex, Without the Copy-Paste — FlineDev
- The Pair — local multi-agent (mentor + executor) setup
- What I learned building an opinionated and minimal coding agent (Pi) — Mario Zechner
- What is the Model Context Protocol (MCP)? — modelcontextprotocol.io

Disclosures

The outline, brainstorming, and modeling were human-directed, with human oversight of AI-model collaboration using Claude Fable 5, Opus 4.8, Sonnet 5, Codex 5.6 Sol, and Gemma 4. Feedback from AI tools was provided by the feynman.is code-audit and source-comparison tools. All output was read aloud and edited by humans for clarity.